

INFORME TÉCNICO: MODELADO DE DOMINIO Y ARQUITECTURA DE PAQUETES UML

PROYECTO FORMATIVO: Eien-System (Gestión Logística y Trazabilidad Transaccional)

PROGRAMA: Análisis y Desarrollo de Software (ADSO)

FICHA: 3336113

APRENDIZ: David Alejandro Meyer Romero

EVIDENCIA: GA4-220501093-AA2-EV01

INDICE

1. INTRODUCCIÓN Y JUSTIFICACIÓN METODOLÓGICA	3
2. DIAGRAMA DE PAQUETES: ARQUITECTURA DEL SOFTWARE	4
Estructura del Diagrama de Paquetes (Eien-System)	4
Relaciones de Dependencia entre Paquetes	5
3. DIAGRAMA DE CLASES DE DOMINIO (CONCEPTUAL)	6
3.1. Aplicación del Principio de Generalización (Herencia)	6
3.2. Especificación Textual de las Clases del Dominio	6
1. Superclase: Persona (Clase Base)	6
2. Subclase: Usuario (Hereda de Persona)	7
3. Subclase: Cliente (Hereda de Persona)	7
4. Clase: Rol	7
5. Clase: Paquete	8
6. Clase: HojaRuta	8
7. Clase: DetalleHojaRuta (Clase de Asociación)	9
8. Clase: Vehiculo	9
9. Clase: EvidenciaEntrega	9
10. Clase: LogAuditoria	10
3.3. Matriz de Justificación de Relaciones y Multiplicidades	11
4. DERIVACIÓN DE OPERACIONES A PARTIR DEL CONTEXTO DE NEGOCIO	18
4.1. Análisis de Caso de Uso Operativo: Registrar Nuevo Paquete	18
Flujo Normal de Operaciones	18
5. CONCLUSIONES DE LA FASE DE MODELADO ORIENTADO A OBJETOS	19

1. INTRODUCCIÓN Y JUSTIFICACIÓN METODOLÓGICA

El presente documento expone el modelado del dominio del negocio para la plataforma **Eien-System**, utilizando el Lenguaje Unificado de Modelado (UML). En la ingeniería de software orientada a objetos, el modelo de dominio es una representación visual de las clases conceptuales u objetos de la vida real que interactúan en el contexto empresarial.

A diferencia del modelo lógico de datos (enfocado en tablas y llaves de persistencia), el diagrama de clases de dominio se centra en el comportamiento, las responsabilidades y las reglas de negocio estructurales, permitiendo cerrar la brecha entre los requerimientos analizados en fases previas y la posterior implementación del código fuente en el backend asíncrono con **FastAPI**.

2. DIAGRAMA DE PAQUETES: ARQUITECTURA DEL SOFTWARE

Se define la estructura lógica del sistema mediante un **Diagrama de Paquetes**. Este artefacto organiza los componentes del software en capas independientes bajo el patrón arquitectónico limpio (*Clean Architecture*), garantizando la mantenibilidad y el bajo acoplamiento.

Estructura del Diagrama de Paquetes (Eien-System):

En la herramienta de software (Visual Paradigm Web), el sistema se subdivide en los siguientes paquetes lógicos:

1. Aplicacion::Nucleo (Paquete de Configuración Global):

- Contiene las configuraciones base de la aplicación, variables de entorno (.env) y la inicialización de módulos de seguridad.
- **Subpaquetes:** Seguridad (Módulo para hashing de contraseñas con **BCrypt** y firma de tokens JWT) y BaseDatos (Sesiones asíncronas con SQLAlchemy para PostgreSQL).

2. Aplicacion::Modelos (Paquete de Dominio / Entidades):

- Contiene las definiciones de las clases base del negocio y sus respectivas correspondencias de objetos en formato **PascalCase**.
- **Componentes internos:** Clases Usuario, Paquete, HojaRuta, Cliente, Vehiculo, EvidenciaEntrega, LogAuditoria.

3. Aplicacion::Esquemas (Paquete de Validación de Datos):

- Encargado de los objetos de transferencia de datos (DTOs) gobernados por **Pydantic** para validar las entradas y salidas de la API REST.

4. Aplicacion::Repositorios (Paquete de Acceso a Datos):

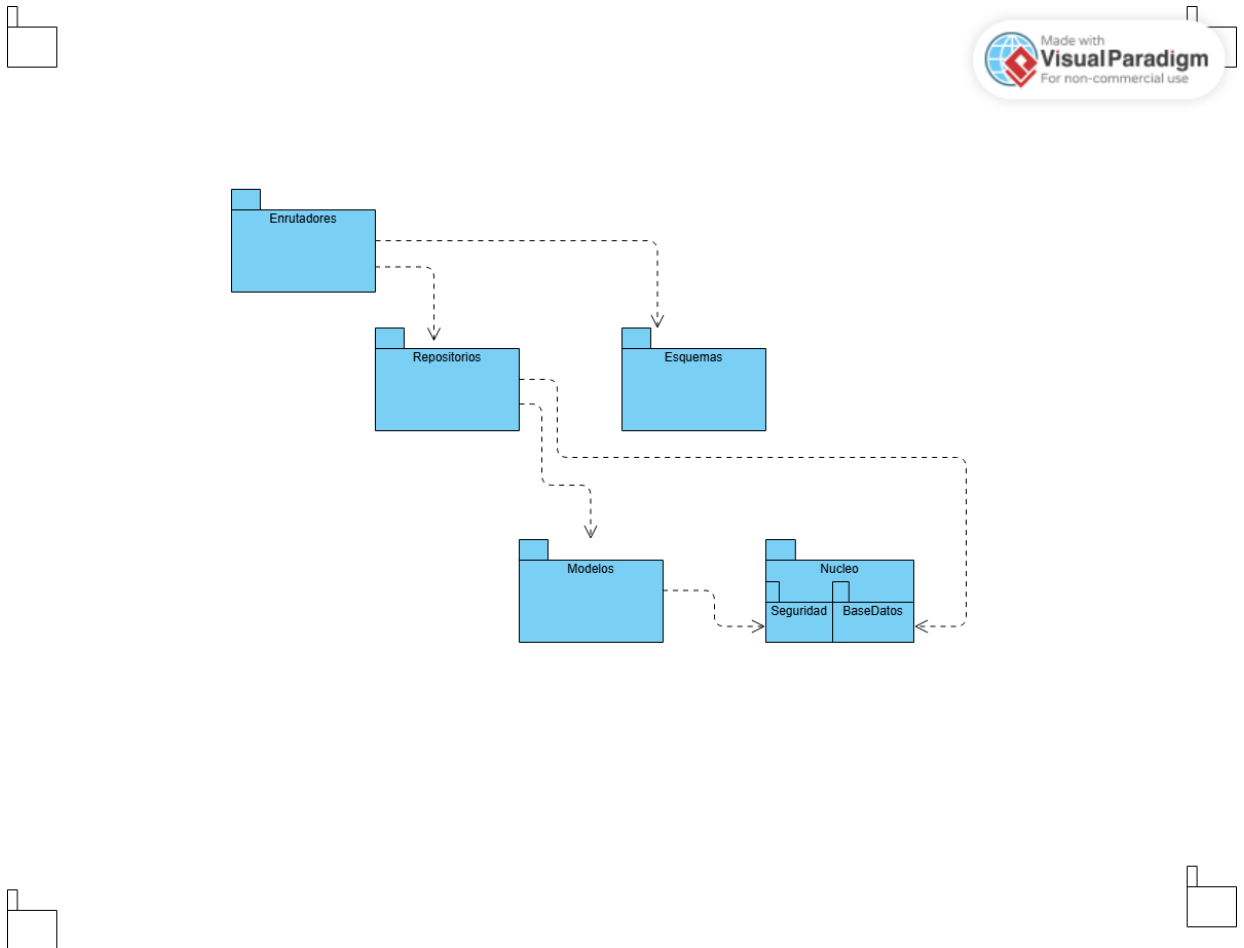
- Encapsula las operaciones CRUD y las consultas lógicas hacia la base de datos de manera aislada.

5. Aplicacion::Enrutadores (Paquete de Controladores / Endpoints):

- Expone las rutas de la API RESTful consumidas por las interfaces web y móviles.
- **Subpaquetes operativos:** RutasAutenticacion, RutasRastreo, RutasDespacho.

Relaciones de Dependencia entre Paquetes:

- Enrutadores → depende de → Repositorios y Esquemas (Para validar peticiones y procesar lógica).
- Repositorios → depende de → Modelos y Nucleo::BaseDatos (Para interactuar con el mapeador objeto-relacional).
- Modelos → depende de → Nucleo::Seguridad (Para aplicar restricciones estructurales de cifrado).



IMG RESOURCE = [Diagrama de Paquetes.png](#)

3. DIAGRAMA DE CLASES DE DOMINIO (CONCEPTUAL)

- **Objetivo:** Representar la estructura conceptual de **Eien-System** mediante los principios de la Programación Orientada a Objetos (POO), implementando encapsulamiento (- para atributos privados), abstracción de operaciones del negocio (+ para métodos públicos) y relaciones jerárquicas avanzadas (Herencia).

3.1. Aplicación del Principio de Generalización (Herencia)

Con el fin de evitar la duplicidad de datos en el diseño de software (Principio DRY) y aumentar la cohesión, se identifica que los empleados del sistema (Usuario) y los remitentes o destinatarios (Cliente) comparten características identitarias idénticas. Por lo tanto, se define la superclase **Persona**, de la cual heredarán ambas clases, optimizando la estructura del código en el backend.

3.2. Especificación Textual de las Clases del Dominio

1. Superclase: *Persona* (Clase Base)

- **Atributos (Privados):**
 - - idPersona : int
 - - documento : String
 - - nombre : String
 - - apellido : String
 - - correo : String
 - - telefono : String
- **Operaciones (Públicas):**
 - + obtenerInformacionCompleta() : String

2. Subclase: Usuario (Hereda de Persona)

- **Atributos (Privados):**

- - claveBcrypt : String
- - estadoUsuario : String

- **Operaciones (Públicas):**

- + estaAutenticado() : boolean
- + validarCredenciales(correo : String, clave : String) : boolean

3. Subclase: Cliente (Hereda de Persona)

- **Atributos (Privados):**

- - direccionFrecuente : String
- - tipoCliente : String

- **Operaciones (Públicas):**

- + verificarHistorialEnvios() : int

4. Clase: Rol

- **Atributos (Privados):**

- - idRol : int
- - nombreRol : String
- - descripcion : String

- **Operaciones (Públicas):**

- + listarPermisosAsociados() : String[]

5. Clase: *Paquete*

- **Atributos (Privados):**

- - idPaquete : int
- - codigoHash : String
- - pesoKg : double
- - dimensiones : String
- - direccionDestino : String
- - prioridad : String
- - estadoActual : String

- **Operaciones (Públicas):**

- + calcularPrioridadAutomatica() : String
- + actualizarEstado(nuevoEstado : String) : boolean
- + generarCodigoRastreo() : String

6. Clase: *HojaRuta*

- **Atributos (Privados):**

- - idHojaRuta : int
- - fechaCreacion : Date
- - estadoRuta : String

- **Operaciones (Públicas):**

- + optimizarSecuenciaParadas() : boolean
- + cerrarRuta() : void

7. Clase: *DetalleHojaRuta* (Clase de Asociación)

- **Atributos (Privados):**
 - - idDetalle : int
 - - ordenEntrega : int
 - - estadoEntregaEspecifico : String
- **Operaciones (Públicas):**
 - + marcarComoVisitado() : void

8. Clase: *Vehiculo*

- **Atributos (Privados):**
 - - idVehiculo : int
 - - placa : String
 - - modelo : String
 - - capacidadMaxKg : double
 - - estadoVehiculo : String
- **Operaciones (Públicas):**
 - + validarDisponibilidadCarga(pesoAdicional : double) : boolean

9. Clase: *EvidenciaEntrega*

- **Atributos (Privados):**
 - - idEvidencia : int
 - - firmaDigitalPath : String
 - - fotoEvidenciaPath : String
 - - coordenadasGps : String
 - - nombreReceptor : String
- **Operaciones (Públicas):**
 - + procesarSincronizacionOffline() : boolean

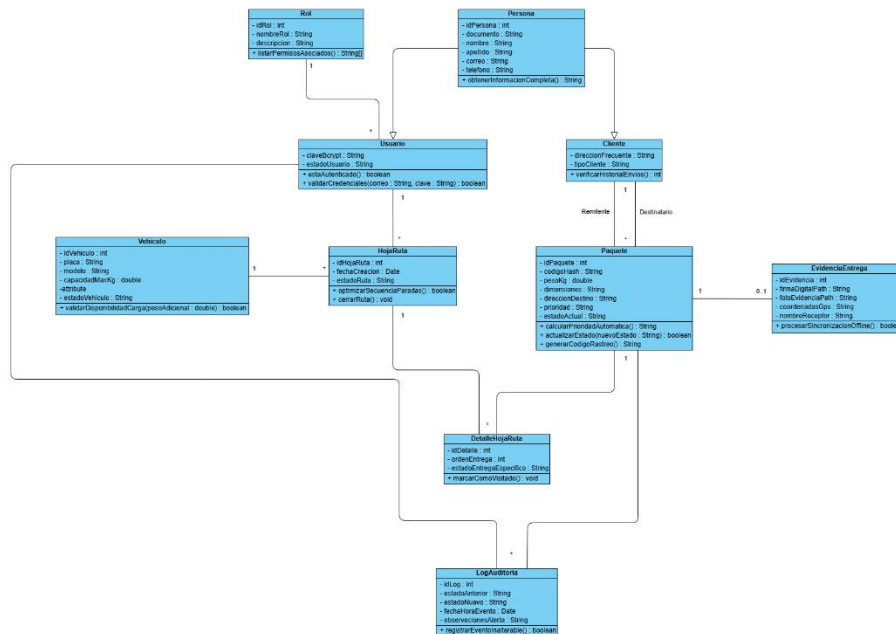
10. Clase: LogAuditoria

- **Atributos (Privados):**

- - idLog : int
- - estadoAnterior : String
- - estadoNuevo : String
- - fechaHoraEvento : Date
- - observacionesAlerta : String

- **Operaciones (Públicas):**

- + registrarEventoInalterable() : boolean



IMG RESOURCE = [Diagrama de dominio v 1.0.jpg](#)

3.3. Matriz de Justificación de Relaciones y Multiplicidades

A continuación, se detalla la lógica de negocio y la cardinalidad asignada a cada una de las asociaciones del modelo de dominio, garantizando la consistencia y la ausencia de ambigüedades en las reglas operativas de la plataforma:

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Rol	Asigna permisos a	Usuario	1 : 0..*	Un rol específico (ej. Conductor, Despachador) puede estar asignado a múltiples usuarios en el sistema, pero cada usuario individual cuenta con un único rol y perfil de acceso activo.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Usuario	Opera como conductor en	HojaRuta	1 : 0..*	Una hoja de ruta logística debe ser gestionada obligatoriamente por un (1) usuario con rol de conductor. Un conductor puede tener asignadas cero o muchas hojas de ruta a lo largo del tiempo.
Vehiculo	Es asignado a	HojaRuta	1 : 0..*	Cada hoja de ruta requiere de un (1) vehículo físico para efectuar el transporte de la carga. Un vehículo de la flota puede ser reutilizado en múltiples hojas de ruta consecutivas.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Cliente	Actúa como Remitente en	Paquete	1 : 0..*	Un paquete es enviado obligatoriamente por un (1) cliente remitente. Un cliente en particular puede registrar y despachar múltiples paquetes hacia el sistema.
Cliente	Actúa como Destinatario en	Paquete	1 : 0..*	Un paquete está consignado a un (1) único cliente destinatario final. Un mismo ciudadano o empresa puede figurar como destinatario en múltiples envíos independientes.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
HojaRuta	Desglosa paradas en	DetalleHojaRuta	1 : 1..*	Una hoja de ruta debe contener obligatoriamente e al menos una (1) parada de entrega registrada en su desglose secuencial, pudiendo albergar múltiples detalles de entrega ordenados de forma eficiente.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Paquete	Es mapeado en	DetalleHojaRuta	1 : 0..*	Un paquete físico se asocia a un registro específico de entrega dentro de una ruta. En términos abstractos, un paquete podría ser reprogramado en más de un detalle si se presentan novedades de entrega.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Paquete	Se valida mediante	EvidenciaEntrega	1 : 0..1	La relación es opcional en su origen. Al registrar la guía, la evidencia no existe (0). Al completarse la entrega física por el módulo <i>Shinrai</i> , se genera y asocia exactamente una (1) evidencia multimedia con firma y GPS.

Clase Origen	Relación / Rol	Clase Destino	Multiplicidad	Justificación Técnica y Regla de Negocio
Paquete	Genera historial en	LogAuditoria	1 : 0..*	Cada cambio de estado de un paquete genera de forma obligatoria un registro cronológico e inalterable para auditoría, acumulando múltiples eventos desde su recolección hasta su entrega final.
Usuario	Gatilla cambios en	LogAuditoria	1 : 0..*	Cada traza de auditoría almacena la identidad del usuario (operario o conductor) que ejecutó la transacción, permitiendo auditar las acciones de múltiples registros de log.

4. DERIVACIÓN DE OPERACIONES A PARTIR DEL CONTEXTO DE NEGOCIO

Para dar cumplimiento a las buenas prácticas de la programación orientada a objetos (POO), las clases del modelo de dominio no solo contienen datos (atributos privados), sino también comportamiento (métodos públicos). Estos comportamientos se deducen directamente de las interacciones del sistema expuestas en las historias de usuario y flujos operativos del proyecto.

4.1. Análisis de Caso de Uso Operativo: Registrar Nuevo Paquete

- **Actor Principal:** Despachador / Operario de Carga.
- **Descripción:** El operario registra una nueva guía física en el sistema de distribución para iniciar su trazabilidad y asignación logística.
- **Precondición:** El usuario debe estar autenticado en la plataforma con el rol adecuado.
- **Postcondición:** El paquete queda creado con un estado de prioridad automático y un código de rastreo único e inalterable en el sistema.

Flujo Normal de Operaciones:

1. El despachador inicia sesión en la plataforma móvil o web, gatillando el método `validarCredenciales()` de la clase `Usuario`.
2. El sistema verifica el nivel de acceso llamando a `listarPermisosAsociados()` desde la clase `Rol` para habilitar el módulo de carga.
3. El operario ingresa las dimensiones, dirección de destino, peso y los datos correspondientes del Cliente (Remitente y Destinatario).
4. La clase `Paquete` procesa los datos y ejecuta internamente el método `calcularPrioridadAutomatica()`, asignando estados de alta prioridad si el peso o la distancia lo requieren.
5. El sistema invoca la operación `generarCodigoRastreo()`, creando el hash criptográfico único para el seguimiento público del paquete.
6. Se guarda el paquete en el paquete lógico de persistencia y de forma simultánea se dispara la operación `registrarEventoInalterable()` dentro de la clase `LogAuditoria`, almacenando la marca de tiempo exacta y el usuario responsable de la creación.

5. CONCLUSIONES DE LA FASE DE MODELADO ORIENTADO A OBJETOS

- **Transición Efectiva Hacia el Desarrollo:** El diseño del diagrama de clases de dominio en formato PascalCase y la especificación de atributos y operaciones con tipos de datos estándar (como String, int, double) facilitan una traducción directa de este modelo conceptual hacia la arquitectura de código asíncrono con **FastAPI** y **Pydantic**.
- **Mantenibilidad y Robustez Mediante Herencia:** La implementación de la superclase Persona reduce significativamente la redundancia de datos (Principio DRY) en la lógica del backend. Cualquier cambio futuro en las reglas de captura de datos ciudadanos (como validaciones de correos o teléfonos) se modificará exclusivamente en la clase abstracta base, afectando positivamente a sus clases hijas.
- **Separación de Responsabilidades:** La organización del software estructurada en el Diagrama de Paquetes en español (Nucleo, Modelos, Esquemas, Repositorios, Enrutadores) garantiza que el sistema cuente con un bajo acoplamiento y una alta cohesión, permitiendo que el equipo de desarrollo trabaje en la lógica de datos de forma totalmente independiente a las interfaces de usuario.